あるべき姿と選定要件

新規：RFPのP7の⑤観点とのひもづけがされていればいい。こういうふうに選べば対応できるというのがわかればいい。

めざすべきデータ基盤を

指すべき姿

DDA戦略が目指すAI活用像→3ゴール。基盤が管理すべきデータ・資産の全体像（何を繋いで、何に使い、どこまで管理するか）を示し、この前提で技術選定に入る

DDA戦略が目指すAI活用の方向性

DDA活用戦略「三方良し」		
ご利用者さま	**社員**	**未来社会**
自立支援、カスタムメイドケア	ムリ・ムダ・ムラ軽減、働きがい向上	需給Gap解消、最適モデル普及

時間軸	データ・AI活用の内容
今	1,000施設のモニタリング（CMC/施設360）、KPIダッシュボード
近い将来	介護記録テキストの自動分析 → ケアプラン改善提案
将来	カメラ映像AI × バイタル × NC の横断分析 → 転倒予防・重度化抑制

→ DDA17プロジェクトの全てが、データ基盤の上に乗る

3ゴール

G1（最優先）

確実な移行と業務継続

土台がないと何も始まらない。219本PLが動く、4モジュール（CMC/モニター/NC/施設360）が止まらない

← 課題 #1, #2, #9, #16, #19

G2（核心）

自分たちで回せる基盤

ベンダー依存では進化が遅い。特別なSEなし、コストが見える・説明できる

← 課題 #4, #6, #12, #14

G3（設計余地）

AI活用を阻害しない基盤設計

「引っ越しただけ」で終わらせない。ロックイン回避、マルチモデル対応、非構造データ枠

← DDA戦略の方向性

データ資産の全体像

分類	データソース	備考
介護記録系	ほのぼの（入居者・ケア記録）、Axist（センサー）、egaku（評価指標）	—
モニタリング系	眠りSCAN（1.92TB）、ナースコール	大容量データの移行計画が必要
外部連携	HD基盤JOY、カメラ映像	カメラ映像は構造化結果のみ取込

主要ユースケース

時期	ユースケース
今回	CMCダッシュボード / 施設モニター / ナースコール分析 / 施設360
将来	介護記録テキスト分析 / カメラ映像AI（構造化結果の横断分析）/ ケアプランAI支援

非構造化データの線引き（3列カラーコードテーブル）

今回スコープ（緑）	設計枠だけ確保（黄）	スコープ外（赤）
構造化データ全量 全量移行 + BI再現	**テキスト（介護記録等）** 将来のNLP/RAG基盤として枠確保（原則4）	**映像生データ** ベンダーSaaSで管理。基盤には入れない

データ管理の考え方

観点	方針
SSOT	新基盤のGold層がKPIの唯一の真実
オーナー	データドメインごとにIT企画部/DDA戦略部が管理
管理範囲	基盤で管理するのは「SOMPOケア固有の資産」。汎用SaaSの生データは外

AI時代のデータ基盤選定

LLMは十分に賢い。鍵は適切なデータを、適切な権限で渡せる基盤であること。

Foundry移行とAI活用の関係

今回のRFPの主目的はFoundry基盤の確実な移行。

しかし移行先の基盤が、将来のAI活用の選択肢を決める。

「今選ぶ基盤」が「将来の AI活用の天井」になりうる。

変わるもの	AI関連の技術スタック。LLMの最適解は半年で変わる。Agent基盤も発展途上。**特定のLLM・ベンダーに絞り込みすぎない設計が重要**
変わりにくいもの	データが正しく溜まり、加工され、探せる状態。ガバナンスが効いていること。**技術が変わっても価値が残る土台**

基盤が拓く AI活用の可能性

1. 介護記録テキストの自動要約・分析
2. カメラ映像のAI解析（転倒検知等）
3. ケアプラン支援（過去データからの最適提案）
4. 全国施設横断のナレッジ共有・検索
5.職員の離職要因分析

SOMPOケアの多数施設・大量の介護データは、AI進化の恩恵を最も受けられる領域

修正する。具体性を出す。RFPにどこまで入れるのかわからないと言われたので、RFPに入れる部分のみ言及し、将来論点は別ページに。各観点の詳細は参照にいれる
一般論として

AI活用基盤に求められる7つの観点

AI活用基盤に必要な7つの観点を、「データ基盤として」「AI活用基盤として」の2軸で整理。まず網羅的な全体像を示し、次ページ（P4）でこの中から今回優先するものを4原則として選定する

観点	データ基盤として	AI活用基盤として
①連携性	ほのぼの/Axist/眠りSCAN/NC接続。CB刷新時にPLが壊れない設計	介護記録・センサーデータをAIが読める形（Gold層）で整備。今のFoundryではAIがデータに触れない
②安全性	700施設×要配慮個人情報の動的RLS。監査ログの法的要件準拠	介護記録をAI分析する際、施設単位・職種単位でアクセス範囲を制御する設計。後付けは困難
③進化性	オープンフォーマット（Delta Lake/Iceberg）でロックイン回避	DDA17PJが様々なAIを使う可能性。1ベンダーのモデルに縛られると17PJ全てが制約を受ける
④実効性	219本PL移行、眠りSCAN 1.92TB移行、4モジュールBI再現	眠りSCAN等のセンサーデータ量でもML処理が性能的に回ること。将来のAI PL追加に耐える設計
⑤運用性	セルフBI（Tableau統一）、カタログ、特別なSEなしで運用	AI実験の記録（誰がいつ何を試したか）が残る仕組みの設計枠。属人化させない
⑥開発速度	Dev/Staging/Prod環境分離、CI/CD	本番に影響なくAI実験ができる環境の分離設計。今のFoundryにはこれがない
⑦規模	1,000施設規模のデータ処理	（今回は具体要件なし。将来の論点として既存資料を参照）

①
②
③
④
⑤
⑥
⑦

※⑦
※将

次に、

連携
整備
統制

データ基盤に紐づく貴社ニーズに合わえた選定要件・原則とすると一般論っぽくならないように

推奨アーキテクチャ・ソフトウェアの採用方針

7観点と資産全体像から判断原則を導き、設計方針を定める

判断原則 → 設計方針

原則1: 統合データ基盤で確実に移行する

← G1 ← ①連携性 + ④実効性

- 219本PySpark PLがデコレーター除去で動くこと
- メダリオン設計（Bronze→Silver→Gold）で構造化+非構造化を同一基盤に
- Dev/Staging/Prod の3環境分離
- バージョン固定 + 計画的アップグレード

満たさないと → PL移行に追加工数、業務停止リスク

原則2: 自分たちで回せる運用

← G2 ← ⑤運用性

- セルフサービスBI（Tableau統一、BI画面の一本化）
- データカタログ（データがどこにあるか探せる）
- 運用に必要なスキルセットの明確化
- 段階的自走ロードマップ（ベンダーからの移管計画）

満たさないと → IT企画部の都度対応が恒常化、属人化の再発

原則3: ガバナンスとコスト透明性

← G1+G2の前提 ← ②安全性

- 700施設×要配慮個人情報の動的RLS
- 監査ログ（法的要件を特定した上での設計）
- コスト構造の全項目開示
- データ量増加時のTCOシミュレーション

満たさないと → 法令違反リスク、予算超過の事後発覚

原則4: 将来の拡張性

← G3 ← ③進化性 + ⑥開発速度

- オープンフォーマット（Delta Lake/Iceberg → ベンダーロックイン回避）
- マルチモデル対応（LLM/MLの選択肢を将来狭めない）
- 非構造化データの設計枠確保（P2の線引きに基づく）

満たさないと → 次の移行が再び困難に、AI活用が基盤制約で頓挫

機能要件の方針

	ベンダーに求める方針
✓	多様なソース対応（PDF/Teams/SF/IoT）
	オープンフォーマット。メダリオン構成。非構造化対応可

	課題 (P1)	→	ゴール (P2)	→	7観点 (P3)
Foundry制約	予告なし変更 PySpark移植	→	G1 確実な移行	→	①連携性 ④実効性
属人化/BBox	セルフBI不可 役割不明確	→	G2 自律運用	→	⑤運用性
コスト不透明	RLS未設計 法的要件未特定	→	G1+G2 の前提	→	②安全性
ロックイン	Foundry固有 形式への依存	→	G3 阻害しない	→	③進化性 ⑥開発速度

つつx

7観点と資産全体像から判断原則を導き、設計方針を定める

判断原則 → 設計方針

原則1（①連携性・④実効性）
多様なデータの統合管理
構造化+非構造化を同一基盤で

原則2（②安全性・⑤運用性）
データ・資産のガバナンス
全資産の可視化・追跡・権限統制

原則3（③進化性）
ロックイン回避
ベンダー交渉力・移行コスト最小化

機能要件の方針

層	ベンダーに求める方針				
	課題 (P1) →	ゴール (P2) →	7観点 (P3) →	4原則 (P4) →	RFPスコープ (P8)
Foundry制約	予告なし変更 PySpark移植 →	G1 確実な移行 →	①連携性 ④実効性 →	原則1 →	PySpark互換PoC （必須）
属人化/BBox	セルフBI不可 役割不明確 →	G2 自律運用 →	⑤運用性 →	原則2 →	スキル要件 自走RM（必須）
コスト不透明	RLS未設計 法的要件未特定 →	G1+G2 の前提 →	②安全性 →	原則3 →	RLS設計, TCO （必須）
ロックイン	Foundry固有 形式への依存 →	G3 阻害しない →	③進化性 ⑥開発速度 →	原則4 →	加点評価

RFPで考慮すべき観点の設計

RFP評価方針案

今回の4原則・FRI評価からRFPで何を求め、同評価し、どう進めるかの方針案を提示

接続実績とかないの当たり前だからいらない

（回答なければ失格）	加点（差別化要素）	将来検討（今回は確認のみ）
ぽの/Axist/眠りSCAN接続実績 刷新（2028年）対応設計	—	HD基盤JOYとの連携 IoTリアルタイム処理
レベルRLS（700施設動的制御） ログ（法的要件準拠）	—	時間軸制御（特定時点のアクセス制限）
プンフォーマット採用 lta Lake / Iceberg）	マルチモデル対応設計 （特定LLMへの非依存）	—
ジュールBI再現（CMC/モニター/NC/施設 SCAN 1.92TB移行計画 本PLのPySpark互換性確認	本番相当データでのPoC実施	—
ーズごとのスキル要件提示 的自走ロードマップ（移管計画）	ナレッジ移転プログラム （ハンズオン研修等）	—
	PoC環境の初期提供 MLflow統合環境の初期設計	—
	—	1,000施設→将来拡張時のスケール対応方針

ACES

② どう評価するか（RFI結果に基づく重み付け）

重み	評価軸	対応原則	根拠
★★★★★	**Foundry移行実現性**	原則1	219本PLが動かなければ他
★★★★★	**運用の自走性**	原則2	ゴール2の核心。特別なSE
★★★★☆	**ガバナンス（法的要件+設計）**	原則3	法的要件準拠の設計なしに
★★★★☆	**コスト構造の透明性**	原則3	経営への説明責任。「作っ
★★★☆☆	**セルフサービスBI**	原則2	IT企画部の瓶頸を解消。R
★★☆☆☆	**将来の拡張性**	原則4	「阻害しない」レベルの確

③ どう進めるか（RFPの出し方）

A案

全ベンダーに自社推奨基盤で提案（幅広い比較）

+ 選択肢が広い。社内説明しやすい

- 各社バラバラの基盤→比較困難。評価コスト大

B案

Databricks前提で提案（実装力比較）

+ 実装力比較に集中。RFIでPySpark互換最高評価の根拠あり

- 「最初から決めていた」と言われる可能性

C案 推奨

A案→結果を見てB案に絞る（2段階）

+ 公正な比較プロセスを踏んだ上でDatabricksに絞れる

+ 社内ガバナンス上、複数基盤を比較した事実が残る

+ A案で予想外の強い候補が出ればそちらも検討可能

+ RFI結果を踏まえれば結果的にDatabricksに収束する蓋然性が高い

ベンダー選定時の RFP評価観点・依頼事項

RFI評価から特に注視すべき評価観点と依頼事項を整理する

RFPで特に確認すべき観点	RFP依頼事項
移行実現性	Foundryパイプラインの移行/ 再構築の進め方、再利用可能範囲、書き換えが必要な範囲、想定工数、類似実績があれば提示すること。あわせてBIツールの選定方針と現行Tableau/Foundry UI資産の再利用/移行方針を記載すること
コスト妥当性	提示条件(※参照)に基づき、初期費用・月額/ 年額費用・従量課金の内訳と算出根拠を提示すること。未提示の前提条件を置く場合は、その想定値と根拠を記載すること。
運用負荷	マネージド/アンマネージド運用、必要に応じてハイブリッド運用の対応可否を示すこと。それぞれについて、役割分担、メリット・デメリット、コスト差分を記載すること。
オープン性	採用するデータ形式(Iceberg/Delta等)を記載すること。標準形式でない部分、制約事項、再構築や追加対応が必要となる箇所があれば明示すること。
カタログ	カタログ管理対象（例：データセット、パイプライン、ダッシュボード、非構造データ）と、リネージの追跡範囲（例：取込元から加工・配信先まで）を記載すること。あわせて、カタログの自動取得 手動設定 / 自動更新の対象範囲・運用方針・制約事項を明示すること。
AI拡張性 **※参考として Sケア・HDのAI検討状況も提示**	AI開発環境環境の構成案、OSS / 業界標準 / 独自製品のどこに依存する構成かと採用理由・実績を記載。環境分離方針、利用可能データ範囲、PoCから本番移行までの前提条件・制約事項を記載すること。採用する各ツールについて、代替可能な選択肢があれば併記すること
LLM柔軟性	利用可能なLLM(gpt/claude等)と制約事項があれば記載すること。
プロジェクト体制	本PJ向けの体制図(役割・人数・主要メンバーの経歴・経験)を記載すること。

※コスト試算前提
Pipeline：219本 / Row Data合計：約2.35TB / Summary Data：約101GB / ダッシュボード：15画面 / BI閲覧者数：約3,000名 / 外部連携：Tableau7、施設300 / 月間アクセス数：約2万アクセス

6社RFI回答のレビュー（AI観点）

原則1・2・3に照らし、AI観点で6社のRFI回答を評価する

方針	評価項目	Deloitte	Fujitsu	PwC	Hitachi	SF	Sky
原則1	非構造化 データ対応	○ Unity Catalog +OpenSearch	△ DI Basic内 具体設計なし	△ Step3先送り	△ Document AI +Cortex Search	△ Data 360 松プラン限定	○ AI Functions+RAG +AI-OCR
原則2	カタログ ・リネージ	○ Unity Catalog 自動リネージ＋品質	○ Unity Catalog DI Basic内蔵	○ DataZone リネージ+MDM	△ Snowflake内蔵 専用カタログなし	△ Data 360 松プラン限定	○ Unity Catalog 松プランのみ
原則2	権限管理	○ Unity Catalog RLS+列マスキング	△ DI Basic内 具体設計なし	○ IAM+Cognito RLS対応	○ Snowflake RLS +マスキング	○ Trust Layer RLS+CRLS	○ IAM（梅竹） Unity Catalog RLS（松）
原則3	AI開発ロックイン	△ Delta Lake/MLflow aws構成での Dify依存に注意	× AI層がKozuchi 独自依存度大	○ AWS標準サービス 移行先あり	△ Snowflake内部形式 移行時は書直し	× SF閉域 全面作直しリスク	○ Delta Lake +Databricks（松）
原則3	マルチLLM	○databricks経由 GPT/Claude/Llama等	× Kozuchi/Takane 独自LLM	△ Bedrock前提 マルチLLM可能	○ Cortex経由 GPT/Claude/Llama等	× Einstein独自 SF内で完結	○ databricks経由 GPT/Claude/Llama等
将来	AI/ML開発環境・ 将来拡張性	○ Agentic AI構成図 MLflow+Registry Dev/Stg/Prod分離 Notebook統合環境	△ DI Basic内蔵 詳細不明 環境分離の具体設計なし	× AI/MLは将来検討扱い 実験環境の提案なし	△ DataRobot MLOps Snowflake内で完結 環境分離方法不明	× 松プランのみ Einstein Studio 独自閉域環境	○ Mosaic AI+MLflow Agent Bricks 梅竹松で段階的環境提供 松でSandbox設計あり

補足：AI開発ロックインの評価基準

× = 独自製品依存（乗り換え＝全面作り直し） … Kozuchi、Einstein等

△ = 特定PF依存（乗り換え可能だが書き直しコスト発生） … Snowflake内部形式等

○ = 業界標準 or OSS（移行先が複数あり現実的に乗り換え可能） … AWS標準、Delta/Iceberg等

6社RFI回答のレビュー（AI以外の観点）

各社のRFI回答をAI以外の観点で評価

評価項目	Deloitte	Fujitsu	PwC	Hitachi	SF	Sky
コスト透明性 /コスト	○（Sケア個別の試算詳細）	△(試算はないが、現時点では不透明なため妥当)	△ (概算のみ)	△（Snowflake従量課金）	△	○（AWS明細型）
既存資産の再利用可能性	○（PySpark native）	○（PySpark native）	○（Glue=PySpark可）	△（Snowpark、PySpark互換だが変換要）	× (PySpark非対応)	○（Glue or Databricks）
ダッシュボード再現性 ※Tableauで要件を満たすか要確認	△（Tableau）	△（PowerBI/Tableau）	△（Tableau推奨）	△（Tableau/PowerBI対応）	△（Tableau）	△（QuickSight）
運用負荷	○（マネージド中心、24/365グローバル保守体制）	△（Databricks主体だが、Sケア主体前提）	×（AWS群の個別管 理）	△（SaaS構成だが複数ベンダー管理）	△	○（保守サービスの明記）
データ形式ロックイン	○（Iceberg/Delta開放）	○（Databricks主体）	△（AWS形式依存）	△（SF API依存）	×（SF API依存）	△（フル AWS構成)
移行リスク	○（3パターン段階）	○（egaku運用実績）	○（3 Step移行）	△（介護業界 Snowflake実績あるが移行計画詳細不足	△（情報不足）	○ (実績多)
セキュリティ	○（Unity Catalog+IAM）	○（IAM+CloudTrail）	○（IAM+Cognito）	○（Horizon + HIPPA準拠 Snowflake実績）	○（Einstein + Tableau RLS）	○（IAM+CloudWatch, 松でUnity Catalog)

6社RFI回答の評価まとめ

RFI時点では、Deloitteが技術面で相対的に有力。
一方で比較継続候補にも固有の強み・確認事項があるため、最終判断は RFPで提案の具体化を求めたうえで行う

会社	現時点の評価	総括	主な懸念/確認事項
Deloitte	有力候補	Sケアデータに基づく詳細試算を提示。 OSSベースでロックイン回避。 AI/ML・非構造化・開発環境すべてに ○評価。24/365運用体制。技術面で穴がない唯一の提案。 ただし運用条件などは要確認	運用条件の具体性（マネージド体制の日本側要員数・対応範囲） Dify等特定ツール依存 (databricks構成推奨なので大きな問題ではない)
Fujitsu	比較継続	egaku/Foundry運用経験者が在籍する唯一の会社。現行環境を知っている価値は大きい。インフラ構成も具体的。「移せるか」の不確実性が最も低い。 ただしKozuchi/DIPのロックインとAI拡張性の弱さが懸念	Kozuchi/DIPの独自ロックイン インフラ構成は具体的だが、 Foundryパイプラインの移行マッピングは未提示
Sky	比較継続	AI/ML構成提案が最も具体的（ Mosaic AI+MLflow+Agent Bricks）。マルチLLM実績も最多。コスト透明性が高い。将来の AI活用を見据えた設計力が強い。 ただし、Tableau非対応がリスク	Tableau非対応（QuickSight固定） 推進体制に具体性がない
Hitachi	比較継続	介護業界の Snowflake実績あり。データ分析民主化の推進計画が具体的。業界理解とユーザー育成の視点は他社にない。 ただしSnowflake単一、PySpark移行の具体計画なし、オープン標準への言及ゼロ	Pyspark移行の工数・実現性 オープン標準への言及ゼロ 統合カタログの言及なし
PwC	比較継続	SOMPOグループとの関係性、ガバナンス設計の具体性、コンサル力は強い。「仕組みづくり」の伴走者としては最適。 ただしAWS単体運用で運用負荷高	運用負荷が高い (AWS郡の個別管理) OSSを明示的に拒否。
SF	優先度低	データ基盤としてはオーバースペック（年 62M～248M円）。AI機能は最高額プラン限定。SF閉域のロックイン。 CRM連携が主目的なら検討余地があるが、全社データ基盤としては不適合	全社データ基盤としてはオーバースペック（年 62M～248M円）。AI機能は松プラン限定。 SF閉域のロックイン

6社RFI回答の評価まとめ

RFI時点では、Deloitteが技術面で相対的に有力。
一方で比較継続候補にも固有の強み・確認事項があるため、最終判断は RFPで提案の具体化を求めたうえで行う

会社	現時点の評価	総括	主な懸念/確認事項
Deloitte	有力候補	Sケアデータに基づく詳細試算を提示。 OSSベースでロックイン回避。 AI/ML・非構造化・開発環境すべてに ○評価。24/365運用体制。技術面で穴がない唯一の提案。 ただし運用条件などは要確認	運用条件の具体性（マネージド体制の日本側要員数・対応範囲） Dify等特定ツール依存 (databricks構成推奨なので大きな問題ではない)
Fujitsu	比較継続	egaku/Foundry運用経験者が在籍する唯一の会社。現行環境を知っている価値は大きい。インフラ構成も具体的。「移せるか」の不確実性が最も低い。 ただしKozuchi/DIPのロックインとAI拡張性の弱さが懸念	Kozuchi/DIPの独自ロックイン インフラ構成は具体的だが、 Foundryパイプラインの移行マッピングは未提示
Sky	比較継続	AI/ML構成提案が最も具体的（ Mosaic AI+MLflow+Agent Bricks）。マルチLLM実績も最多。コスト透明性が高い。将来の AI活用を見据えた設計力が強い。 ただし、Tableau非対応がリスク	Tableau非対応（QuickSight固定） 推進体制に具体性がない
Hitachi	比較継続	介護業界の Snowflake実績あり。データ分析民主化の推進計画が具体的。業界理解とユーザー育成の視点は他社にない。 ただしSnowflake単一、PySpark移行の具体計画なし、オープン標準への言及ゼロ	Pyspark移行の工数・実現性 オープン標準への言及ゼロ 統合カタログの言及なし
PwC	比較継続	SOMPOグループとの関係性、ガバナンス設計の具体性、コンサル力は強い。「仕組みづくり」の伴走者としては最適。 ただしAWS単体運用で運用負荷高	運用負荷が高い (AWS郡の個別管理) OSSを明示的に拒否。
SF	優先度低	データ基盤としてはオーバースペック（年 62M～248M円）。AI機能は最高額プラン限定。SF閉域のロックイン。 CRM連携が主目的なら検討余地があるが、全社データ基盤としては不適合	全社データ基盤としてはオーバースペック（年 62M～248M円）。AI機能は松プラン限定。 SF閉域のロックイン

XXX

7つ観点詳細版 ①連携性

社内外のデータソースと確実に繋がり、ソース側の変更にも追随できること。接続できなければデータ基盤は機能せず、CB（介護記録システム）刷新のたびにパイプラインが壊れる構造では自走できない。

識すべきこと

基盤として	AI活用基盤として
の/Axist/眠りSCAN/NC接続。CB（介護記録システム）刷新が壊れない設計	介護記録・センサーデータをAIが読める形（Gold層）で整備。今のFoundryではAIがデータに触れない

Oケア固有の要件

ぼの（NDソフトウェア）	入居者基本情報、ケア記録、請求データ。APIまたはDB直接接続。最重要ソース。
（パナソニック）	センサーデータ（離床・行動検知等）。リアルタイム性は今回不要、バッチ取込で対応。
SCAN（パラマウントベッド）	睡眠・バイタルセンサー。累積1.92TB。現行はFoundry経由で取込中。最大データソース。
スコール	通知ログ。施設ごとにシステムが異なる可能性あり。接続方式の標準化が必要。
u	評価指標データ。定期バッチ取込。
盤JOY	ホールディングスとの共有データ。連携方式は要調査（Sケア側判断）。
新対応	介護記録システム改定時にスキーマ変更を自動検知し、パイプラインが壊れない設計が必須。現行Foundryはこの仕組みが不十分。

RFP評価基準

必須（GO/NO-GO）

- 上記6ソースへの接続実績または具体的な接続計画の提示
- CB刷新時のスキーマ変更検知・自動通知の仕組みの提案

加点

今回は加点項目なし

将来検討

- HD基盤JOY 双方向連
- IoTセンサーのリアルタイム
ング取込

将来の論点

IoTセンサーのリアルタイムストリーミング取込（Kafka等）。HD基盤JOYとの双方向データ連携と権限設計。

7つ観点詳細版 ②安全性

要配慮個人情報を扱う介護データに対し、適切なアクセス制御と監査が実現できること。後付けでは実装困難な領域であり、設計段階での要件定義が不可欠。

	AI活用基盤として
人情報の動的RLS（行・列レベル）。監査ログの 護保険法・個人情報保護法）	介護記録をAI分析する際、施設単位・職種単位でアクセス範囲を制御する設計。後付けは困難。AI経由でのデータ参照も制御対象に含める

要件

	290施設（介護付きホーム）+ 146棟（サ高住）+ 628事業所（在宅）= 1,000+拠点。全拠点の動的RLSが必要。
	要配慮個人情報（介護記録、バイタル、服薬記録、入居者マスタ等）。漏洩時のリスクは最高レベル。
元	施設×職種×データ種別×時間。例: 施設長Aは施設Aのデータのみ参照可。本部スタッフは全施設横断。ミドル部門は集計済みKPIのみ。
	誰がいつ何のデータにアクセスしたか。介護保険法・個人情報保護法に準拠した保持期間の設計が必要。法的要件の特定が未了（課題#10）。
	監査ログが肥大化しストレージコスト増大。法的に必要な保持期間が未特定のため、削除ルールが設定できていない。

RFP評価基準

必須（GO/NO-GO）

- 動的RLS（行・列レベル）の設計提案（Unity Catalog等）
- 1,000拠点規模での動的RLS性能テスト計画
- 監査ログ保持期間の法的要件調査と設計提案

加点

今回は加点項目なし

将来

- 時
タ
- AI
と

将来の論点

AIモデルが学習に使うデータの権限管理。AI出力の監査（誰がどのモデルで何を分析したか）の仕組み設計。

7つ観点詳細版 ③進化性

技術の世代交代が激しいAI分野で、基盤が特定技術に縛られず進化できること。Foundryで経験した「ロックイン」の再発防止がSOMPOケアにとって特に重要な観点。

	AI活用基盤として
a Lake/Iceberg）でロックイン回避。基盤を変えてもデータが再利用可能な	DDA17PJが様々なAIを使う可能性。1ベンダーのモデルに縛られると17PJ全てが制約を受ける。マルチモデル対応の設計枠を確保する

oundry固有形式（オントロジー等）への依存により移行が困難になった。同じ轍を踏まない設計を明示的に求する。

PJが異なるAI技術（LLM、CV、時系列予測等）を使う可能性がある。基盤が特定MLフレームワークやLLMロバイダーに依存すると、17PJ全てが制約を受ける。

ータをDelta Lake / Iceberg / Parquet等の標準形式で保存。基盤を変えてもデータが再利用可能。特定ベンーのクローズド形式を使わない。

RFP評価基準

必須（GO/NO-GO）

- データ保存形式がDelta Lake / Iceberg / Parquet等のオープンフォーマットであること

加点

- マルチ
ームワ
計）

将来の論点

LLMの進化速度に基盤が追随できるかの継続評価。マルチクラ

7つ観点詳細版 ④実効性

設計上可能なだけでなく、SOMPOケアの実データ・実規模で実際に動くこと。「できます」という提案ではなく、「やりました」「やってみせます」を証明させる観点。

	AI活用基盤として
互換)、眠りSCAN 1.92TB移行、4モジュー 施設360）のBI再現	眠りSCAN等のセンサーデータ量でもML処理が性能的に回ること。将来のAIパイプライン追加に耐える設計

Foundry固有デコレーター（@transform_df, @configure等）の除去が必要。デコレーター除去後にそのまま動作するか、代表的なPLでのPoC実施を求める。
最大のデータソース。移行時間・コストの見積もりと、移行完了後のデータ検証（件数・整合性）計画が必要。
CMCダッシュボード、施設モニター、ナースコール分析、施設360。現行Foundry Workshop UIをTableauで再現。全KPIの一致を確認するQAプロセスを含めること。
Foundry Rules（Java）コードの移行可能性。移行不能な場合の代替設計の提示が必要。
現行は手動展開。新基盤ではCI/CDで自動化。300施設への自動展開の実証が求められる。

RFP評価基準

必須（GO/NO-GO）

- 219本PLのうち代表的なPLの移植PoC実施（Foundryデコレーター除去後の動作確認）
- 眠りSCAN 1.92TB規模でのデータ移行計画（移行時間・コスト・検証手順）
- 4モジュールBI再現の設計提案とQAプロセス

加点

- 本番相当環境でのPoC実施（
ックスではなく実データ規模
- 300施設自動展開のデモまた
示

将来の論点

データ量が年4%成長した場合の5年後の性能予測。AIパイプライン（MLflow等）が

7つ観点詳細版 ⑤運用性

特別なSEを抱えなくても、IT企画部・DDA戦略部が日常運用を自走できること。石郷岡氏が4/1 MTGで最重要事項として明示した観点。

	AI活用基盤として
ー)、データカタログ（データの所在 用できるスキル要件の明確化	AI実験の記録（誰がいつ何を試したか）が残る仕組みの設計枠を確保。属人化させない実験管理基盤

別なSEがいなくても回せる」（4/1 MTGで明確に指摘）。7観点の中にこの観点が欠落していたことを正式
完。

ル部門のデータ抽出依頼にIT企画部が都度対応。セルフサービス化でこの依存を断ち切る。Tableau + デ
カタログによるセルフBI環境の実現が必須。

面がFoundryとTableauの2系統に分散。新基盤ではTableauに統一し、二重管理を解消する。

と外注の役割分担が不明確。運用体制の明確化（どこまで内製でどこからベンダー委託か）を契約段階で
する。

運用は2名程度（データエンジニア中心）。ベンダーは高度案件のみ。段階的に移管し、1年後には日常運
内製化する計画が必要。

タがどこにあるか探せる・見つけられる仕組み。Unity Catalog等で実現。メタデータ管理・データリネー
可視化を含む。

RFP評価基準

必須（GO/NO-GO）

- 日常運用に必要なスキルセットの明示（SQL以外に何が必要か）
- 想定体制・人数の提示（内製何名、ベンダー何名）
- ベンダーからの段階的移管計画（マイルストーン付き）

加点

- ナレッジ移転プ
グ計画）の具体

将来の論点

LLM活用による非エンジニア（ミドル部門）の対応範囲拡大。自然言語

7つ観点詳細版 ⑥開発速度

新しいAIユースケースの検証を素早く回せる環境があること。本番に影響を与えずにAI実験ができる環境の分離が、DDA17PJの実行速度を左右する。

	AI活用基盤として
CI/CDによる安全なリリースフロー	本番に影響なくAI実験ができる環境の分離設計。現行Foundryにはこれがなく、開発時の資材が本番に残存する問題が発生している

ト環境と本番環境が未分離。開発・検証用の資材（データ、コード、設定）が本番に残存するリスクが継的に発生。

（開発）/ Staging（検証）/ Prod（本番）の3環境。CI/CDパイプラインで安全にリリース。各環境のコス見積もりも含めること。

護記録のテキスト分析等、将来のPoCを本番データに影響なく試せる環境。実験の再現性確保（MLflow。

RFP評価基準

必須（GO/NO-GO）

環境分離は④実効性で必須として要求済み。本観点での追加必須項目なし。

加点

- PoC環境の
  クス）
- MLflow等の
  の再現性・

将来の論点

AIモデル実験のサンドボックス環境の本格整備。特徴量ストア（F

7つ観点詳細版 ⑦規模

AI活用が進み、複数のAIモデル/エージェントが同時に動く時代のガバナンスとリソース管理。現時点では具体要件がないため今回はスコープ外とするが、設計段階で意識しておく観点。

	AI活用基盤として
模で対応可能。特別な	今回は具体要件なし。将来的に複数AIモデルが同時にデータにアクセスする場面が想定されるが、現時点ではスコープ外

。DDA17PJのうち今回基盤に乗るユースケースは限定的であり、規模面での特別な課題は現ない。

時にデータにアクセスし、競合・コスト急増・権限混在が発生するシナリオ。現時点では想段階で考慮しておくと移行コストを抑えられる。

RFP評価基準

必須（GO/NO-GO）

今回は必須項目なし

加点

今回は加点項目なし

将来の論点

AIエージェントの増加に伴うアクセス制御・リソース管理の複雑化。モデル/ユースケー

AI活用に向けた将来論点

AI活用の進展を見据え、将来追加で検討が必要となる論点を7つの観点で整理する

観点	将来論点	将来の検討事項	いつ検討すべきか
①連携性	HD/グループ会社とのデータ共有や、施設 IoT・映像等のリアルタイム連携が必要になる可能性がある	共有方式、リアルタイム連携要否、対象データ、更新頻度、遅延許容、障害時再送	HD/グループ会社側の連携方針が確定した時、またはIoT/映像ユースケースが具体化した時
②安全性	AIがデータアクセス主体となる場合、人と同等の ID・権限・監査・ガードレール設計が必要になる	AI主体を含む認可方針、データ分類、監査方針、AI個別権限、入出力制御	AIユースケース検討時
③進化性	AIモデルは短期間で変化するため、精度評価・劣化検知・改善サイクルを前提とした運用が必要になる	評価指標、劣化検知、モデル /プロンプト/RAG構成の管理、切替手順、改善サイクル	本番利用するAI機能・モデルの導入方針が決まった時
④実効性	介護記録テキスト、音声、 IoT、映像等の非構造化データは、構造化データと異なる加工設計が必要になる	対象データ選定、原本 /加工後データ管理、チャンク化・ベクトル化、品質管理、構造化データとの紐付け	非構造化データを用いる具体ユースケースが決まり、対象データと精度要件が明確になった時
⑤運用性	AIはモデル・データ・プロンプト・ツールの複合体となるため、障害時の原因切り分け、停止判断、コスト管理が難しくなる	ログ/トレース、障害切り分け、停止基準、コスト可視化、監視・運用体制	AIユースケースの実装設計に着手する時
⑥開発速度	AI開発は試行錯誤前提であり、 PoC→本番→横展開の標準手順が未整備だと立ち上げが遅延する	標準運用、利用可能データ範囲、権限付与、PoC→本番移行条件	PoC候補具体化時に環境要件を確定
⑦規模	AI/Agent増加で品質・責任・コスト管理が難しくなる	登録管理、標準化、責任分界、コスト管理	複数AI機能が開発された時

※ 体制・人材・制度の論点も別途存在

SW構成基本方針/データ・フロー基本方針

主要コンポーネントの基本構成方針

AI活用の進展も見据えつつ、機能だけではなく運用実現性も考慮した方針を採用する

7観点との紐づけ

コンポーネント	構成方針
インテグレーション	•多様なデータソースを標準的な接続方式で取り込める構成とする •既存連携の再利用を重視しつつ、将来の非構造化データや外部サービス連携の追加にも対応可能とする
ストレージ	•構造化データを中心に、原本・標準化済み・活用用データを統合管理できる構成とする •将来の非構造化データ追加を阻害しないことを前提とする
オーケストレーション	•ワークフロー実行、スケジュール管理、再実行、監視を一元的に扱える構成とする •個別運用を増やしすぎず運用負荷を抑えやすい構成とする
変換・処理	•既存パイプライン資産の再利用性・移植性を重視し、処理ロジックを継承しやすい構成とする •将来の追加加工 (非構造データ含む)にも拡張可能とし、管理しやすい構成とする。個別実装や属人運用を増やさないことを重視する
ガバナンス	•カタログ、リネージ、権限、監査ログを統合的に管理・運用できる構成とする •データとパイプラインを横断して可視化・統制でき、管理対象、更新責任、監査運用が継続できることを重視する
ML/AI	•ML/AI機能そのものは現時点で存在しないため、実装可能な枠確保を基本とする •将来のML・AIモデル、プロンプト /RAG、推論ログを追加しやすい構成とする •将来的に特定モデルのみや特定技術に限定されない構成とする
可視化	•既存BI資産の継続活用は必須としないが、移行後に可視化のクオリティや活用面で負担がない表示が可能な構成とする

データの段階的な階層設計

Bronze（生データ）→ Silver（クレンジング済）→ Gold（ビジネスロジック適用）の 3層構造

設計基準	Bronze層	Silver層	Gold層
目的	**ソースデータの保管 「事実の記録」として不変**	**分析可能な状態に標準化 クレンジング・正規化済**	**ビジネス KPI・BI/ダッシュボード接続用 集計・結合・ロジック適用済**
変換ルール	取り込んだままを保持	型変換・NULL処理・重複排除 カラム名の標準化	集計・結合・ビジネスロジック
品質チェック	到着確認・件数チェック・スキーマ確認	品質ゲート（違反行はアラート）	KPI整合性チェック
テーブル命名例	raw_care_records raw_sensor_data	clean_care_records master_facilities	kpi_facility_monthly dim_staff, fact_care_events

設計原則：各層の独立性を維持し、 Bronze → Silver → Goldの一方向依存を徹底

階層に分離することで不具合時の原因特定を明確化。 BIツール利用ユーザーには Gold層を提供、分析ユーザーには Silver層を提供。

開発・本番環境の分離

安定運用のための環境分離の方針

3階層の環境構成

開発環境 (dev)	ステージング環境 (stg)	本番環境 (prod)
開発者個人の実験・検証	**統合テスト・品質検証**	**本番環境・ダッシュボード接続**
Branch単位で作業	本番同等スキーマで検証	変更はStagingからのリリースのみ
サンプルデータを使用	CI/CDで自動デプロイ	本番データ・施設別アクセス制御

原則：環境間のデータ移動は禁止。コード（パイプライン定義）のみが　CI/CDで環境間を移動する

Dev環境では特定時点のスナップショットのサンプルデータを使用して、開発に活用。　Stagingで本番同等の環境で検証を行い、合格した場合に　Prodへリリース。

開発～リリースの流れ

① 開発・実験
開発環境

- Feature Branchで作業
- サンプルデータで開発・検証
- テスト実行

② PR・レビュー
コードレビュー

- PR作成（最低1名承認）
- テスト自動チェック
- mainブランチへMerge

③ ステージング反映・検証
ステージング環境

- CI/CDで自動デプロイ
- 本番同等データで統合テスト
- データ品質チェック通過を確認

④ 本番リリース PR
本番リリースのレビュー

- リリース承認（担当者）
- CI/CDで自動デプロイ

Appendix 目次

別紙A アーキテクチャ・技術選定

A-1 アーキテクチャ: DWH+DL vs Lakehouse
A-2 テーブルフォーマットとは
A-3 UniFormによるフォーマットの収束
A-4 プラットフォーム比較: Databricks vs Snowflake vs AWS
A-5 本プロジェクトにおける各 PF評価
A-6 BI層にSnowflakeを挟む必要性
A-7 オーケストレーション: 構成パターン比較
A-8 データ変換: Foundry PySpark移植戦略
A-9 セマンティックレイヤー: Ontology代替

別紙B ガバナンス移行マッピング

B-1 データガバナンス: Foundry機能の代替マッピング

別紙C 推奨技術スタック総括

C-1 推奨技術スタック: 総括
C-2 技術選定サマリー

1 アーキテクチャ:「 DWH + Data Lake」vs「Lakehouse」

従来の2層構成を1つに統合した Lakehouse が新しい標準

従来型: DWH + Data Lake（2層構成）

Data Lake
S3 / GCS
生データを安価に保管
SQLで直接分析は不可

→ ETL →

Data Warehouse
Snowflake / Redshift
加工済みデータをBI向けに提供
コスト高・ML/AI非対応

- ✗ データが2箇所に存在 → 同期コスト・整合性の問題
- ✗ DWHライセンス + Lake運用 = 二重課金
- ✗ ML/AIはDWHから取り出して別環境で実行が必要

Lakehouse（統合型 — 推奨）

Lakehouse
Databricks + S3 / Palantir Foundry（現行）

保管	分析	加工	AI	管理
Data Lakeの安価なストレージ	DWHの高速なSQL検索	ETLパイプラインを統合実行	ML/AIを同一基盤で	権限・監査リネージ統合

- ✓ データは1箇所 → 管理がシンプル・整合性の問題なし
- ✓ ストレージはS3価格 + コンピュートは従量課金
- ✓ BI・ML/AI・ETLが同一基盤で完結

観点	DWH + Data Lake（2層）	Lakehouse（統合）
データの場所	2箇所（Lakeに保管 → DWHにコピー）	1箇所（S3上にすべて統合）
コスト構造	DWHライセンス + Lake運用 = 二重課金	ストレージはS3価格 + コンピュート従量課金
BI対応	◎ DWH側は非常に高速	○ 近年大幅改善。実用上十分
ML/AI対応	△ DWHから取り出して別環境で実行	◎ 同一基盤でデータに直接アクセス
ガバナンス	○ DWHとLakeで別々に管理	◎ 統合管理（権限・監査・リネージ）
ベンダーロックイン	△ DWH側にロックインされやすい	◎ オープンフォーマット（iceberg等）で回避可能
市場トレンド	レガシー構成	新規構築の主流

2 テーブルフォーマットとは

S3上のファイルを「テーブル」として扱うための仕組み

テーブルフォーマットなし（S3のみ）

S3バケット

- data/year=2024/month=01/part-001.parquet
- data/year=2024/month=02/part-002.parquet
- data/year=2024/month=03/part-003.parquet
- ...

→ 単なるファイル置き場

→ SQLでの検索・更新・削除が不可能

テーブルフォーマットあり（Iceberg/Delta）

S3 + メタデータ層

✓ スキーマ管理 — 列の追加・変更を安全に実行

✓ ACIDトランザクション — 途中で壊れない書き込み保証

✓ Time Travel — 過去のデータに遡れる（監査対応）

✓ パーティション管理 — 高速検索のための自動分割

✓ 行レベルの更新・削除 — RDBのようなUPDATE/DELETE

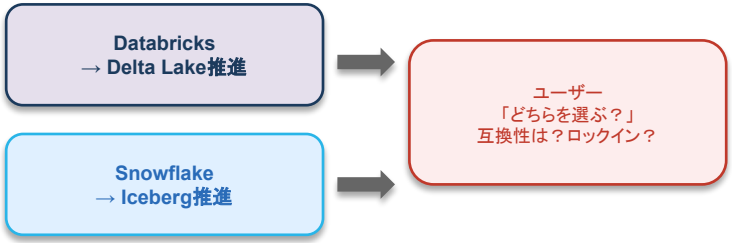
主要フォーマット 3種の位置づけ

	Apache Iceberg	Delta Lake	Apache Hudi
起源	Netflix → Apache財団	Databricks → Linux Foundation	Uber → Apache財団
位置づけ	業界標準の互換フォーマット	Spark環境での最高性能	ストリーミング特化（ニッチ）
採用状況	975社以上（2026年） Snowflake/Athena/BigQuery対応	Databricksエコシステム中心 Spark上での性能は最速	採用企業は限定的 リアルタイムUpsertに強い
相互運用	マルチエンジン対応が最も広い ベンダー中立性◎	UniFormでIceberg互換を実現 Databricks利用時のデフォルト	Iceberg互換を自ら追加 本PJでは選択肢外

2 補足：UniFormによる「フォーマット」の収束

Delta Lakeで書き込み → 自動でIcebergメタデータ生成 → 外部エンジンからIcebergとして読取り可能

以前の状況：二者択一



現在：UniForm（2024年6月GA）で解決

UniFormの制約事項（要認識）

制約	内容	影響度
読み取り専用	外部エンジンからの書き込み不可。書き込みはDatabricks/Delta経由のみ	中
メタデータの遅延	Icebergメタデータは非同期生成。書き込み直後は一瞬のタイムラグあり	低
性能ペナルティ	Snowflakeからの読み取りはネイティブ比で約20%の性能低下	中
Databricks固有機能	UniFormはオープンな業界標準ではなくDatabricks固有の機能	要認識

結論：Databricks利用時：Delta Lake + UniForm（Iceberg互換）を推奨。

Databricksを書き込みプラットフォームとする前提で二者択一が不要に。マルチプラットフォーム時：Icebergを標準化。ベンダー中立性が最も高い。

3 プラットフォーム比較：Databricks vs Snowflake vs AWS Native

別紙A-4

PySpark互換性・AI拡張性・ガバナンスの観点で評価

Databricks
データエンジニアリング + AI の統合環境

Snowflake
SQLの王者。シンプルで速い

AWS Native
部品を自分で組み立てる最安構成

評価軸	Databricks	Snowflake	AWS Native
PySpark互換性	◎ そのまま動く	× Snowparkへ書き換え	○ EMR/Glueで動く
AI/ML	◎ MLflow, Vector Search等 統合	△ Cortex AI 限定的	○ SageMaker+Bedrock
SQL分析性能	○ 十分高速	◎ 最速クラス	○ Athena/Redshift
ガバナンス	◎ Unity Catalog統合管理	◎ Horizon（SF内は強い）	△ Lake Formation
オープン性	◎ UC(コアOSS), Delta+Iceberg	△ 独自形式+Apache Polaris	○ OSSベース
運用負荷	○ サーバーレス化進行中	◎ フルマネージド	× 自前構築・運用
コスト構造	中～高（DBU+インフラ二重）	中（クレジット制）	低～中（従量+人件費）
必要人材	2名～（データエンジニア）	1-2名（SQL人材）	3-4名（インフラ+データ）

3 本プロジェクトにおける各プラットフォームの評価

Foundry移行の要件に照らした適合度

Databricks
◎ 最も適合度が高い

Snowflake
△ 本プロジェクトでは不利

AWS Native
○ 安いが運用が重い

評価軸	Databricks	Snowflake	AWS Native
PySpark互換性	◎ そのまま動作 （デコレーター除去のみ）	× Snowparkへ全面書き換え RDD/UDF/ML全て移植が必要	○ EMR上で動作可能 ただしGlue PySpark は制約あり
AI/ML拡張性	◎ MLflow, Vector Search Model Serving, Feature Store	△ Cortex ML は発展途上 Databricksに比べ機能限定的	○ SageMaker連携可能 ただし統合度は低い
ベンダーロックイン	○ Unity Catalog OSS化 で懸念が大幅に軽減	× 独自形式でデータ保持 エクスポートにコスト発生	◎ OSSベース S3 + Iceberg で中立性高い
運用負荷	◎ 統合プラットフォーム Foundryに最も近い体験	◎ フルマネージド 運用負荷は最小	× 6-8個のサービス統合 障害切り分けが複雑
人材確保	○ Spark/Python人材 市場に比較的多い	○ SQL中心で敷居は低い ただしSnowpark人材は少数	△ AWSインフラスキル 全サービス横断の人材が困難
コスト	○ 中程度 Commit割引で最適化可能	△ 高め PySpark書き換え工数も加算	○ 最安 ただし人件費含むと差は縮小

結論：本プロジェクトでは Databricksを推奨

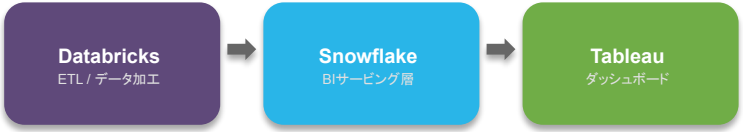
PySpark互換性、AI拡張性、Unity Catalogのオープン性が、Foundry移行の要件に最も合致する。
SnowflakeはPySpark書き換えコスト大、AWS NativeはOSS運用工数が見えにくいリスクあり。

3 BI層にSnowflakeを挟む必要性

よくあるパターン（Databricks ETL + Snowflake）は必要か？

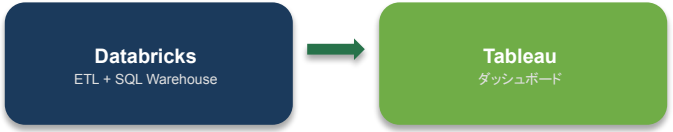
以前はTableau同時多数接続に Snowflakeを挟むメリットがあったが、現在は性能差は実運用では少なく、コストと手間だけ重くなる設計。

パターンA：Snowflakeを挟む構成



⚠️ Snowflake追加コスト：年間 1,500-3,000万円（ライセンス ＋ 同期パイプライン構築・運用）

パターンB：直接接続（推奨）



✓ シンプル。追加コストなし。年間 1,500-3,000万円を削減

ポイント：「登録ユーザー数」と「同時接続数」は全く異なる

指標	推定値	根拠
登録ユーザー	800 - 1,000人	施設長700名 + 本部スタッフ
日次アクティブ率	20 - 30%	施設長は毎日閲覧しない。週次月次チェックが多い
ピーク同時接続	40 - 80人	朝の業務開始時、月初レポート確認時が最大
平常時同時接続	10 - 30人	—

Databricks SQL Serverlessの同時実行能力

10並行クエリ/クラスター × 自動スケーリング
ピーク80人 → 8クラスターで対応（自動）
上限: 1,000並行クエリ — 全く問題なし
2025年: 実ワークロード 5倍高速化を達成

さらにTableau Extractで同時接続問題自体が消える

4ダッシュボードはいずれも日次バッチ更新で十分
Extract: 定期的にデータをTableau Serverにキャッシュ
800人がアクセスしてもDatabricksへの負荷はゼロ
Snowflakeを挟む理由が完全になくなる

4 オーケストレーション：構成パターンの比較

Databricks中心であればLakeflow単体が最もシンプルかつ推奨。外部連携の複雑度に応じて構成を選択

パターン1

Lakeflow 単体 (databricks)

推奨

Lakeflow Jobs
- バッチETL
- ストリーミング
- ML推論

Lakeflow Pipelines
- 宣言的ETL
- 品質チェック

パターン2

Lakeflow + Airflow

外部連携が複雑な場合

Lakeflow Jobs
- Databricks内ETL

Airflow DAG
- 外部API連携
- クロスシステム依存

パターン3

Airflow/Dagster 一本化

統一管理を重視

外部オーケストレーター
- 全ジョブ統合管理

Databricks API呼出
- Spark Job投入のみ

評価軸	パターン1：Lakeflow単体	パターン2：Lakeflow + Airflow	パターン3：外部一本化
運用負荷	◎ 最小（フルマネージド）	△ 2ツール運用が必要	△ 外部ツール運用が増加
Unity Catalog統合	◎ ネイティブ統合	○ Databricks内は統合	△ API経由のみ
外部システム連携	△ 複雑な場合に不便	◎ Airflowで柔軟に対応	◎ 全システム一元管理
セットアップ	◎ 追加構築不要	△ Airflow環境の構築が必要	△ 専用環境の構築が必要
コスト	◎ Databricks内で完結	○ Airflow運用コスト追加	○ 外部ツールコスト追加

結論：Databricks中心であればLakeflow単体を推奨

フルマネージドで運用負荷が低く、Unity Catalogとの統合も深い。外部システム連携が複雑な場合のみAirflowを追加。

5 データ変換: Foundry PySpark パイプラインの移植戦略

既存のPySparkコードはDatabricksへほぼそのまま移植可能。追加ツールの導入は不要

移植の全体像

現行: Foundry

@transform_df
@configure
Foundry固有デコレーター
PySpark DataFrame API

→ デコレーター除去のみ

移行先: Databricks

PySpark DataFrame API
（そのまま動作）
Lakeflow Pipelines
Databricks Notebooks

Databricksが標準提供する機能（追加ツール不要）

IDE / 開発環境	**Databricks Notebooks + SQL Editor** Foundry Code Repositoriesの代替
ジョブ管理	**Lakeflow Jobs（サーバーレス）** Foundryビルドシステムの代替
宣言的ETL	**Lakeflow Declarative Pipelines** パイプライン定義・自動実行
CI/CD	**Databricks Asset Bundles + GitHub Actions** Code RepositoriesのPR→ビルドの代替
リネージ	**Unity Catalog（PySpark列レベル対応）** Foundry自動リネージの代替
テスト	**Databricks DLT Expectations** データ品質チェックの自動化

Foundry → Databricks: 機能対応マッピング

Foundry機能	Databricksでの対応	移植工数	備考
PySpark パイプライン（@transform_df等）	PySpark on Databricks	低 デコレーター除去のみ	コアロジックはそのまま動作
Code Repositories（Git + ビルド一体化）	GitHub + Databricks Asset Bundles + Lakeflow Jobs	中 CI/CDパイプライン構築	RFP提案にこの工数が含まれているか要確認
ブランチベースのデータ開発	スキーマ分離 + 開発/本番 プロモーションフロー	中～高 運用フロー再設計	Foundry最大の差別化機能。完全代替は困難 開発→本番の運用ルール再設計が必要
自動データバージョニング	Iceberg/Delta Time Travel + VACUUMポリシー設計	低 保持期間の設計のみ	「全ビルド無期限保持」は非現実的 スナップショット保持ポリシーの設計が必要
自動データリネージ	Unity Catalog Lineage （PySpark列レベル対応）	なし 自動で動作	.toPandas()やRDD操作のみ非対応

6 セマンティックレイヤー: Ontology代替の要否

ビジネス用語の定義・計算ロジックを一箇所で管理する層。FoundryのOntologyに相当

セマンティックレイヤーなし

- **Tableau** — 入居率 = 入居者数 / 定員（定義A）
- **AIチャット** — 入居率 = 入居者数 / ベッド数（定義B（微妙に違う））
- **Excelレポート** — 入居率 = ???（定義C（不明））

→ 数値が合わない。信頼を失う

セマンティックレイヤーあり

セマンティックレイヤー
入居率 = 入居者数 / 定員（唯一の定義）

- Tableau
- AIチャット
- Excel

→ すべて同じ数値。Single Source of Truth

必要性の判断基準

ユースケース	セマンティックレイヤー	理由
ダッシュボード4つのみ（現状維持）	不要（後から追加可能）	Tableauの計算フィールドで十分
ダッシュボード増加（10+）	推奨	計算定義の食い違い防止
AIチャット（自然言語分析）	実質必須	セマンティック層によりAIのSQL精度が約3倍向上 （学術研究 16%→54%）
予測AI + 自動レポート	必須	Feature Storeとの連携も必要

主な選択肢:

- **Databricks Metric Views** UC統合・GA済み
- **dbt MetricFlow** OSS (Apache 2.0)
- **Cube.dev** OLAP高速化

結論：初期構築では不要。AIチャットをスコープに入れる場合は初期から設計が必要。

Databricks利用時はMetric Views（Unity Catalog内蔵、2025年後半GA）が最もシンプルな選択肢。

7 データガバナンス: Foundry機能の代替マッピング

Foundryで統合的に動作していたガバナンス機能を、Unity Catalogでどう代替するか

Foundry機能	役割	Unity Catalogでの代替	対応度	ギャップ / 注意点
Markings（タグ自動伝播）	データ分類タグの自動伝播	Tags（手動 or API設定）	△	自動伝播なし— CI/CDでタグ付け ルールを自動化する運用設計が必要
RLS（行レベルセキュリティ）	施設ごとのアクセス制御（700施設）	行フィルタ+ 列マスキング	◎	対応可能。動的RLSも実装可 Foundryと同等のきめ細かい制御
自動リネージ	データの来歴を自動追跡	SQL: 自動追跡 PySpark: 自動追跡（GA済）	○	PySpark lineageはUnity Catalogで 自動追跡。toPandas/RDDは非対応
監査ログ	誰が何にアクセスしたかの記録	System Tables （Audit Logs）	◎	対応可能。SQLで柔軟に分析可 Foundryより分析の自由度が高い
PII検出（Sensitive Data Scanner）	個人情報の自動検出・マスキング	Presidio（OSS） or AWS Macie	△	別途導入・設定が必要 介護データは要配慮個人情報— 必須

Unity Catalogの位置づけ

コアカタログ機能はApache 2.0でOSS化。ただし高度なガバナンス（列リネージ、動的ポリシー、PII分類）はDatabricks商用版が必要。

◎ そのまま代替可能

- RLS
- 監査ログ

○ 運用設計で対応

- Markings
- リネージ

△ 別途ツール導入

- PII検出

8 推奨技術スタック：総括

Foundry移行における推奨構成

BI / 分析	Tableau（既存維持）
セマンティック	初期は不要 → AI導入時 Metric Views等を検討
データ変換	PySpark（Python）+ dbt Core（SQL）
オーケストレーション	Lakeflow Jobs（+ 必要時 Airflow）
ガバナンス	Unity Catalog（データ + ML + AI 統合管理）
テーブルフォーマット	Delta Lake + UniForm（Iceberg互換）
ストレージ	S3 + Lakehouseアーキテクチャ

基盤：Databricks Lakehouse Platform

この構成のメリット

1. **PySpark移植コスト最小**
   Foundryデコレーター除去のみでほぼ動作
2. **ツール数最小**
   Databricks + dbt + Tableau の3つが中核
3. **ロックイン回避**
   UC(OSS) + Delta+UniForm(Iceberg互換)
4. **AI拡張性**
   MLflow, Vector Search, Model Serving統合済み
5. **運用人材**
   2名程度（データエンジニア中心）で運用可能

リスク・注意点

- DBU単価の不透明性 — 見積段階で開示を要求
- Markings自動伝播の欠如 — CI/CD自動化で補完
- ストレージ膨張 — VACUUMポリシーの設計必須